

test_apple_container

Revision: 1 **Last modified:** 2026-06-13T00:00:00Z **Authority:** vasic-digital tmux project **Maintainer:** milosvasic **Scope:** §11.4.18 script companion doc for scripts/test_apple_container.sh

Overview

scripts/test_apple_container.sh makes "test tmx on Linux, in a container, under macOS" **real and reproducible**. On a macOS host it boots a genuine Linux VM via Apple's native container runtime, **builds the Linux tmux 3.6a ELF binary inside that VM** (so it is a real Linux build — osdep-linux.o + libjemalloc.so.2 + libevent_core, not a Mach-O), then runs the project's own test suite against that binary and captures real PASS / FAIL / SKIP evidence.

It is the macOS-host sibling of scripts/test_containerized.sh (which uses podman/docker on a Linux host). Per §11.4.81 cross-platform parity, this is the macOS-equivalent path for the same "verify the Linux build + runtime" intent.

The host is Darwin/arm64; the container kernel is Linux/aarch64. That OS gap is the whole point — it is captured verbatim in uname.txt.

Prerequisites

- macOS host (the script SKIPS cleanly with exit 3 on any other OS).
- Apple container ≥ 1.0.0 installed (brew install container) **and** the system running (container system start → container system status reports running).
- A Linux kernel + a base image available to the runtime. The default image is docker.io/library/ubuntu:22.04; pull it once with container images pull docker.io/library/ubuntu:22.04 (or let the first run fetch it).
- The tmux/ submodule populated (git submodule update --init tmux) — the script SKIPS if tmux/configure is absent.
- Host tools: bash, tar, curl, gunzip, awk, sed. Optional but recommended: go (for the Linux tmx-state-bin cross-build — without it, tests 27/38/43 surface tmx-state as non-functional on Linux). gtimeout (coreutils) is used if present; a pure-bash watchdog is the fallback.

No network is required **inside** the container — see "Internal behaviour → Offline build-deps". Network **on the host** is required the first time (to fetch the apt .deb build dependencies) unless the container VM itself has outbound connectivity.

Usage examples

```
# Standard run – build + topology-dispatched suite, auto-cleanup.
bash scripts/test_apple_container.sh

# Keep the container + scratch dir for debugging.
bash scripts/test_apple_container.sh --keep

# Re-run the suite against an already-built container (after --
  keep).
bash scripts/test_apple_container.sh --keep --no-build

# Different base image.
bash scripts/test_apple_container.sh --image docker.io/library/
  ubuntu:24.04
```

Environment overrides:

Variable	Default	Effect
TMX_AC_IMAGE	docker.io/library/ ubuntu:22.04	base Linux image
TMX_AC_TIMEOUT	60	per-container op timeout (seconds)
TMX_AC_BUILD_T0	600	build / suite step timeout (seconds)

Exit codes

Code	Meaning
0	Suite ran; FAIL=0 — real success
1	Suite ran; ≥ 1 FAIL — real product defect (investigate, do not mask)
2	Build failed inside the container — toolchain/source defect
3	SKIP: not macOS, or Apple container / kernel absent / not running

Outputs

Captured under docs/qa/2026-06-13-apple-container/linux-run/:

File	Content
uname.txt	in-container <code>uname -s -m</code> → Linux aarch64
tmux-version.txt	in-container <code>tmux -V</code> → tmux 3.6a (the built binary)
elf-proof.txt	ELF magic (7f 45 4c 46) + ldd (jemalloc/event/tinfo/libc)
build.log	full in-container <code>build_native.sh</code> transcript

File	Content
run_all.log	full in-container suite transcript
summary.txt	PASS/FAIL/SKIP counts + per-test verdicts + SKIP reasons

Internal behaviour

One long-lived container + exec

Apple container pays a large start-up penalty when a `--mount type=virtiofs` is attached (≈ 55 s), and mounting the whole repo (large `.git` + submodules + build trees) **hangs**. So the script boots **one** long-lived container (`sleep infinity`, no mount, ~ 2 s start-up) and runs each step via container `exec`. Source is delivered by **tar-stream** (`tar -cf - ... | container exec -i ... tar -xf -`), never a mount.

Offline build-deps (§11.4.77 regeneration mechanism)

The minimal container VM frequently has **no outbound network** (common when the macOS host sits behind a split-tunnel VPN whose resolver is on `127.0.0.0/8` — the `vmnet` gateway cannot route the VM's traffic). The script detects this (`raw / dev/tcp/1.1.1.1/443` probe) and, instead of failing, runs an offline toolchain install:

1. Fetch the apt Packages indexes **host-side** (the host has network).
2. Push them into the container's `/var/lib/apt/lists`.
3. Resolve the **full dependency closure offline** via `apt-get install --print-uris` (apt does the transitive resolution).
4. Download the resolved `.deb` files host-side.
5. `dpkg -i` them in a multi-pass loop + `dpkg --configure -a` (settles Pre-Depends ordering — e.g. `python3-minimal` $\rightarrow$ `python3.10-minimal`).

If the container VM *does* have network, the fast path (`apt-get update && apt-get install ...`) is used instead.

Linux `tmx-state-bin` cross-build (§11.4.81 parity)

`scripts/tmx-state-bin` on the host is a macOS **Mach-O** binary; copied into Linux it yields `Exec format error`. The script cross-compiles the `scripts/tmx-state/` Go source for the container's arch on the host (`G00S=linux G0ARCH=...` `CGO_ENABLED=0 go build` $\rightarrow$ static ELF) and overwrites the leaked Mach-O, so the `cwd-persistence` tests run against a real Linux binary.

§11.4.3 topology dispatch (which tests SKIP and why)

The minimal container VM has **no user systemd bus** (`pid 1` is `sleep`, no `systemd-run/systemctl`), **no host npm CodeGraph CLI**, **no real TTY / mouse**, and **no SSH remotes**. Tests that assert those host-only or repo-tooling facts are emitted as

SKIP-with-reason by an explicit topology table in the harness — never a FAIL against an environment that structurally cannot satisfy them (a §11.4.1 FAIL-bluff), and never a fake PASS. The SKIP families:

- **systemd-cgroup** (08, 09, 12, 13, 14, 15, 24, 40): the tmx Linux isolation primitive is `systemd-run --user --scope`; absent here.
- **host-tooling** (20, 21, 22 CodeGraph; 41 pandoc/weasyprint render).
- **repo-wiring** (18: `.gitmodules` constitution SSH submodule entry).
- **interactive-shell** (43: `cwd-persist` fires from an interactive bash `PROMPT_COMMAND`; non-interactive `exec` has no prompt cycle).
- **physical-terminal** (44–48, 55–59: real TTY / SGR-mouse / clipboard).

Tests **not** in the table run for real; their genuine PASS/FAIL/SKIP is reported verbatim (e.g. 16 self-SKIPs on the documented tmux/Linux rename-hook race; 39 self-SKIPs because EUID 0 bypasses mode bits; 51 self-SKIPs when the `cmd/workable-items` Go binary is not built).

Cleanup

A `trap ... EXIT INT TERM` stops + removes the container and deletes the host scratch dir on **every** exit path (unless `--keep`). Every container call is timeout-guarded so a wedged runtime cannot hang the script.

Edge cases

- **Wedged runtime.** If a prior container `rm/run` hung, the runtime can wedge (a fresh boot hangs at "Starting container"). Recover with `container system stop && container system start`. The harness's timeout guards prevent it from *adding* to a wedge, but cannot un-wedge a pre-existing one.
- **No host network on first run.** If neither the container nor the host can reach `ports.ubuntu.com`, the `offline-deps` step FAILs (exit 2) with a clear message — there is no cached `.deb` set bundled in-repo.
- **No go on host.** `tmx-state` tests (27/38/43) surface `tmx-state` as non-functional on Linux rather than crashing; install `Go ≥ 1.21` to exercise them.

Related scripts

- `scripts/test_containerized.sh` — Linux-host (podman/docker) sibling.
- `scripts/build_native.sh` — the Linux build branch this script invokes inside the container.
- `scripts/tests/run_all.sh` — the suite whose per-test classification this script mirrors (with topology dispatch added).
- `scripts/tmx.template` — the `tmx` wrapper generated in-container.

Sources verified 2026-06-13

- Apple container project + CLI semantics (`container run/exec/system`, `virtiofs` mount cost, `vmnet` gateway): <https://github.com/apple/container>

- Debian/Ubuntu offline .deb install + Pre-Depends ordering and apt-get install --print-uris resolution: man dpkg, man apt-get (Ubuntu 22.04 jammy, on-host).
- Go cross-compilation (GOOS/GOARCH/CGO_ENABLED): <https://go.dev/doc/install/source#environment>

Last verified

2026-06-13 — real run on Darwin 24.5.0 / arm64 host with Apple container 1.0.0:
container uname = Linux aarch64, built tmux 3.6a (ELF + jemalloc), suite result
PASS=30 FAIL=0 SKIP=28.